



Darshan UNIVERSITY

Python Programming - 2301CS404

Lab - 14

Roll No.: 310

Name: Ronit Bhadania

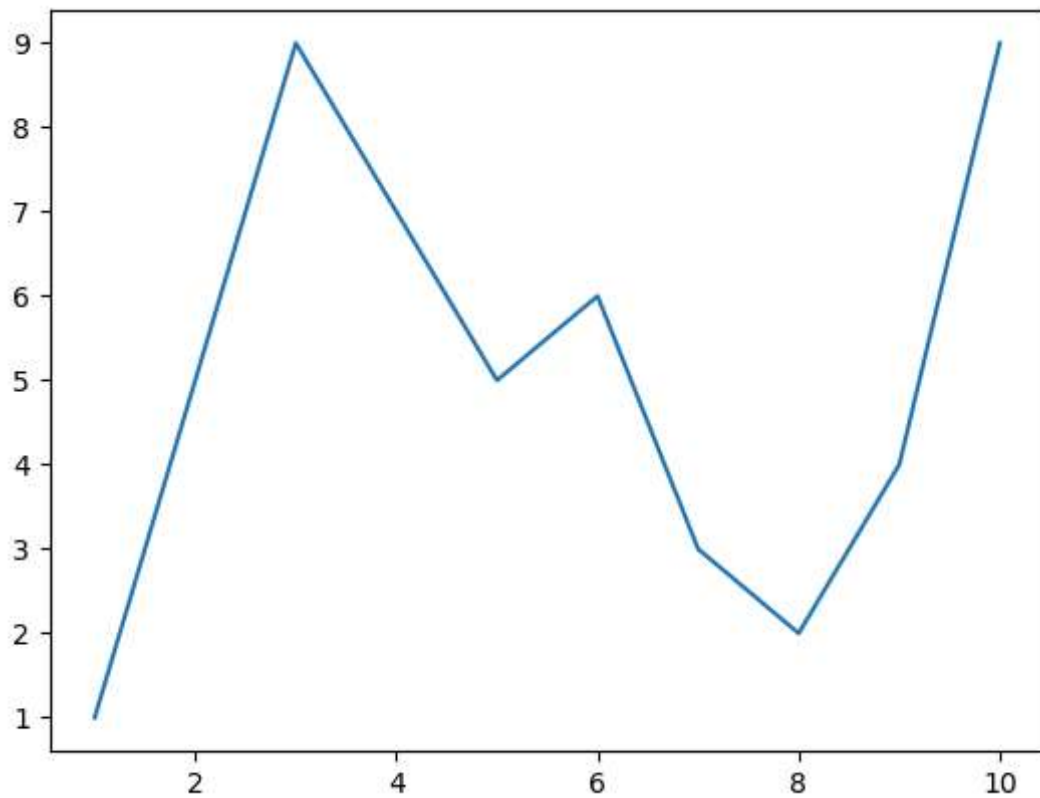
Import matplotlib below

```
In [1]: import matplotlib.pyplot as plt
```

01) WAP to display the line chart for given data for x and y axis.

```
In [5]: x = range(1,11)  
y = [1,5,9,7,5,6,3,2,4,9]
```

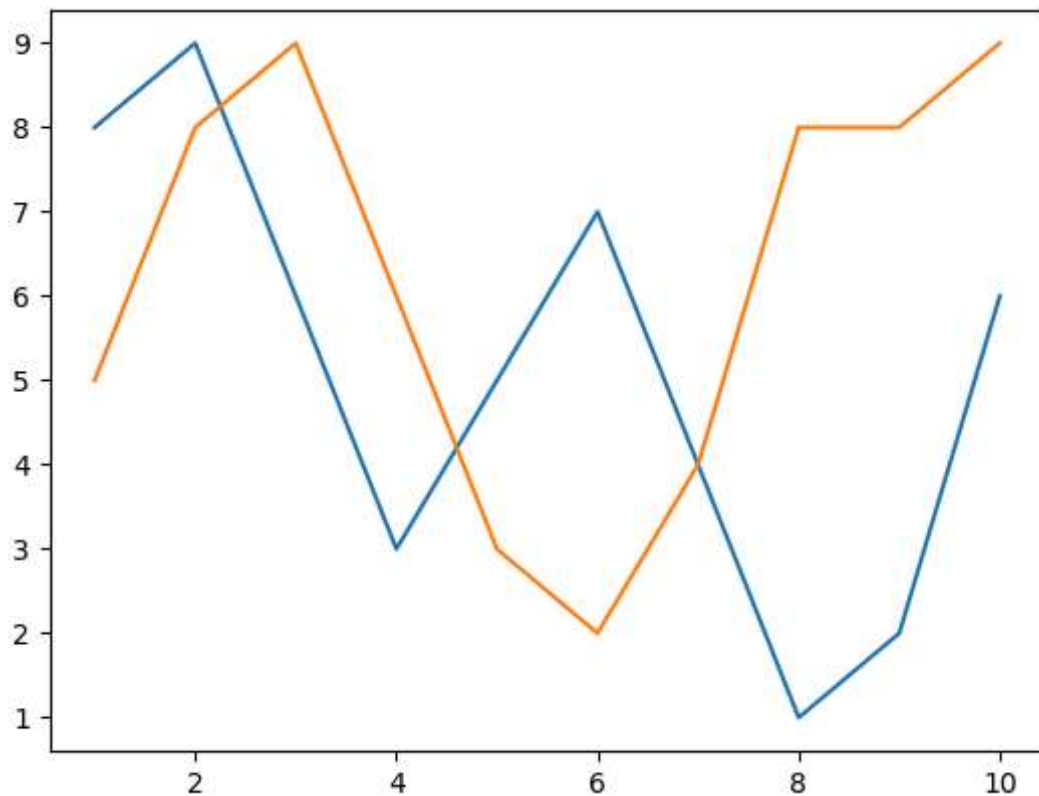
```
In [6]: plt.plot(x,y)  
plt.show()
```



02) WAP to generate line chart for given data.

```
In [16]: x = range(1,11,1)
cxMarks= [8,9,6,3,5,7,4,1,2,6]
cyMarks= [5,8,9,6,3,2,4,8,8,9]
```

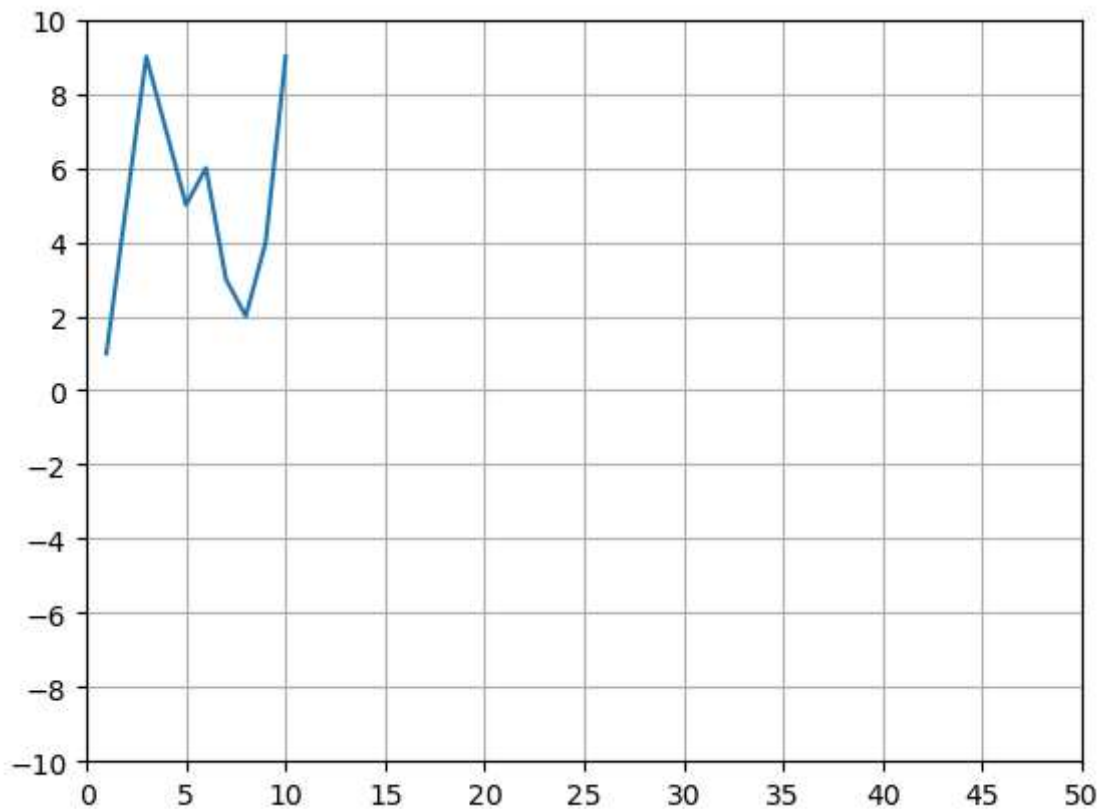
```
In [17]: plt.plot(x,cxMarks)
plt.plot(x,cyMarks)
plt.show()
```



03) WAP to generate line chart for given data, but create your custom axis.

```
In [18]: x = range(1,11)
         y = [1,5,9,7,5,6,3,2,4,9]
```

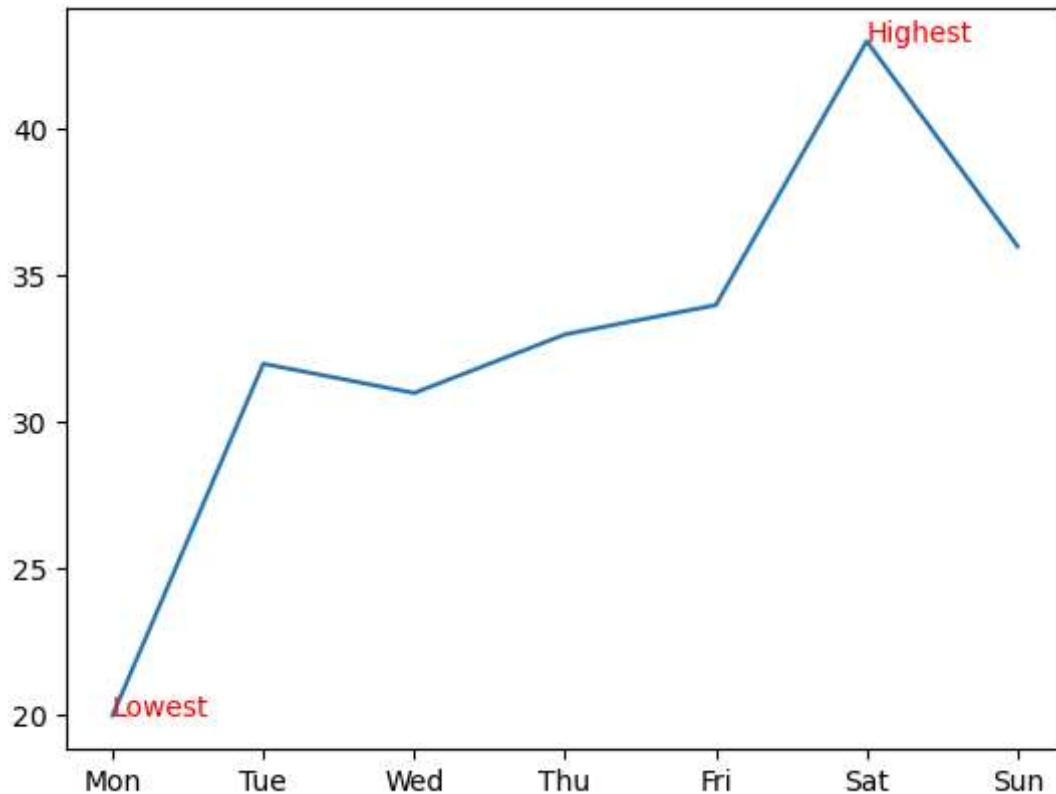
```
In [19]: ax = plt.axes()
         ax.set_xlim([0,50])
         ax.set_ylim([-10,10])
         ax.set_xticks([0,5,10,15,20,25,30,35,40,45,50])
         ax.set_yticks([-10,-8,-6,-4,-2,0,2,4,6,8,10])
         ax.grid()
         plt.plot(x,y)
         plt.show()
```



04) WAP to plot a line chart showing temperature variation during a week. Mark the lowest and highest temperature.

```
In [21]: Days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
          Temperature = [20, 32, 31, 33, 34, 43, 36]
```

```
In [22]: plt.plot(Days, Temperature)
plt.annotate('Lowest', xy=['Mon', 20], c='r')
plt.annotate('Highest', xy=['Sat', 43], c='r')
plt.show()
```



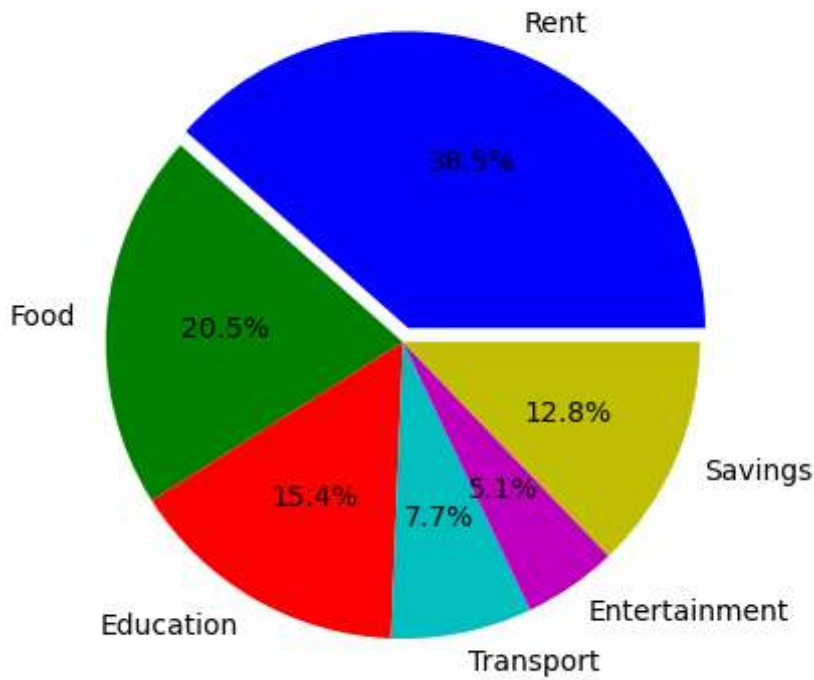
05) WAP to create a detailed pie chart representing the monthly expenditure distribution of a family.

The chart should display labels, percentage values, title, different colors for each slice, and explode one slice to highlight the highest expense.

```
In [27]: Categories = ["Rent", "Food", "Education", "Transport", "Entertainment", "Saving"]
Amount = [15000, 8000, 6000, 3000, 2000, 5000]
```

```
In [ ]: c=['b','g','r','c','m','y']
e=[0.05,0,0,0,0,0]
plt.pie(Amount,colors=c,labels=Categories,explode=e,autopct='%1.1f%%')
plt.title('Monthly expenditure distribution of a family')
plt.show()
```

Monthly expenditure distribution of a family

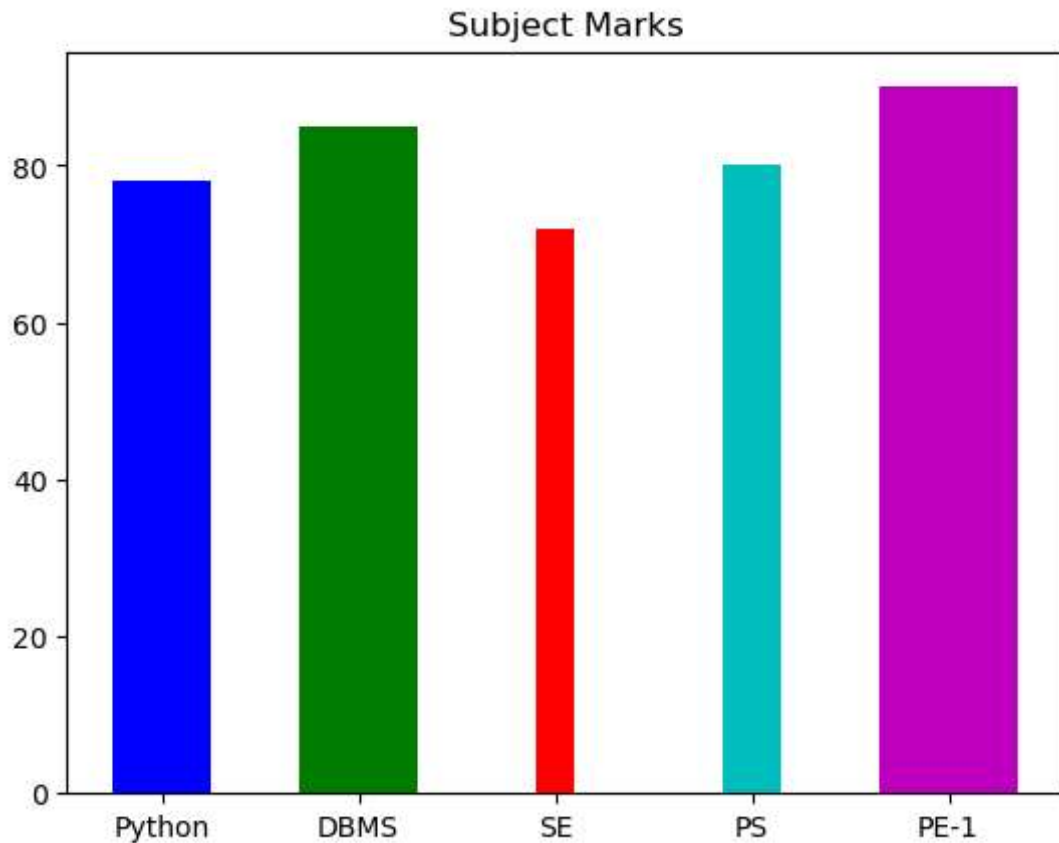


06) WAP to create bar chart showing the marks obtained by students in different subjects.

The program should also display the marks on the top of the respective bar.

```
In [1]: Subjects = ["Python", "DBMS", "SE", "PS", "PE-1"]
Marks = [78, 85, 72, 80, 90]
```

```
In [12]: l=['1st','2nd','3rd','4th','5th']
c=['b','g','r','c','m']
w = [0.5,0.6,0.2,0.3,0.7]
plt.title('Subject Marks')
plt.bar(Subjects,Marks,color=c,label=l,width=w)
plt.show()
```

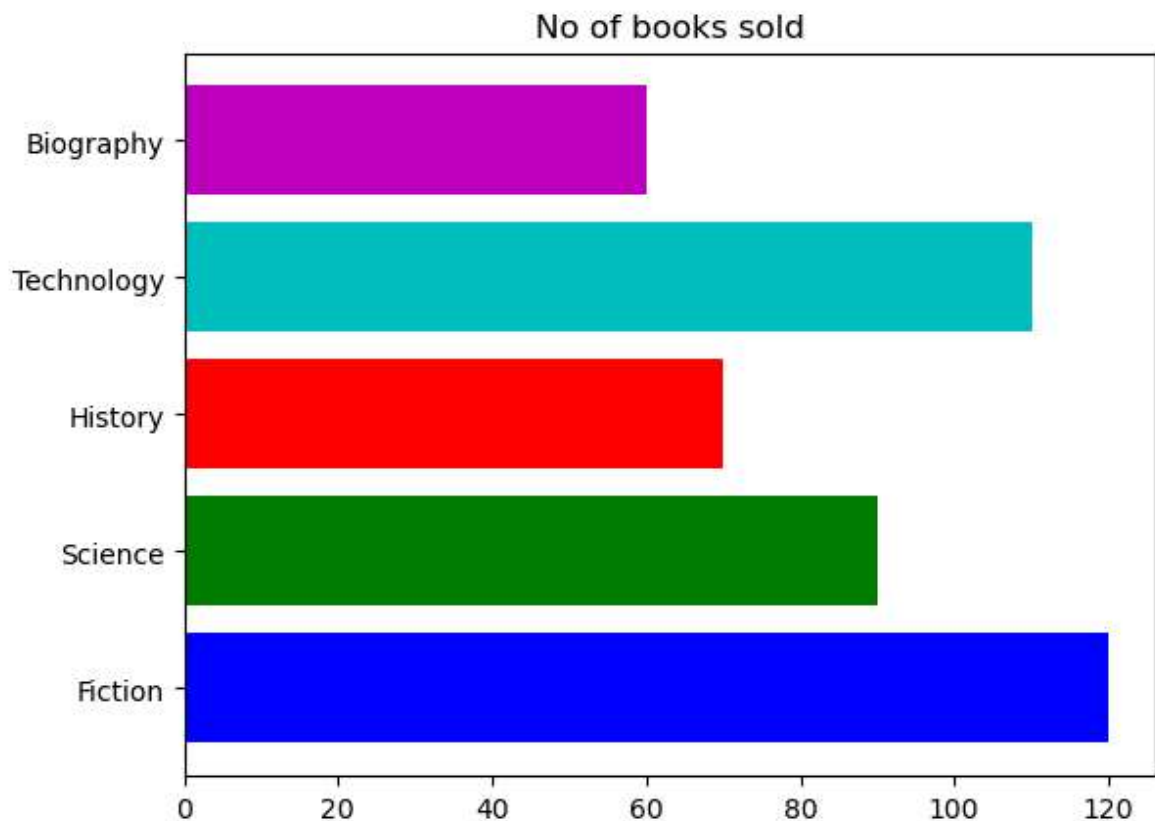


07) WAP to create a horizontal bar chart representing the number of books sold in different categories.

The program should also display the number of books sold at the end of the respective bar.

```
In [11]: Categories = ["Fiction", "Science", "History", "Technology", "Biography"]  
Books_Sold = [120, 90, 70, 110, 60]
```

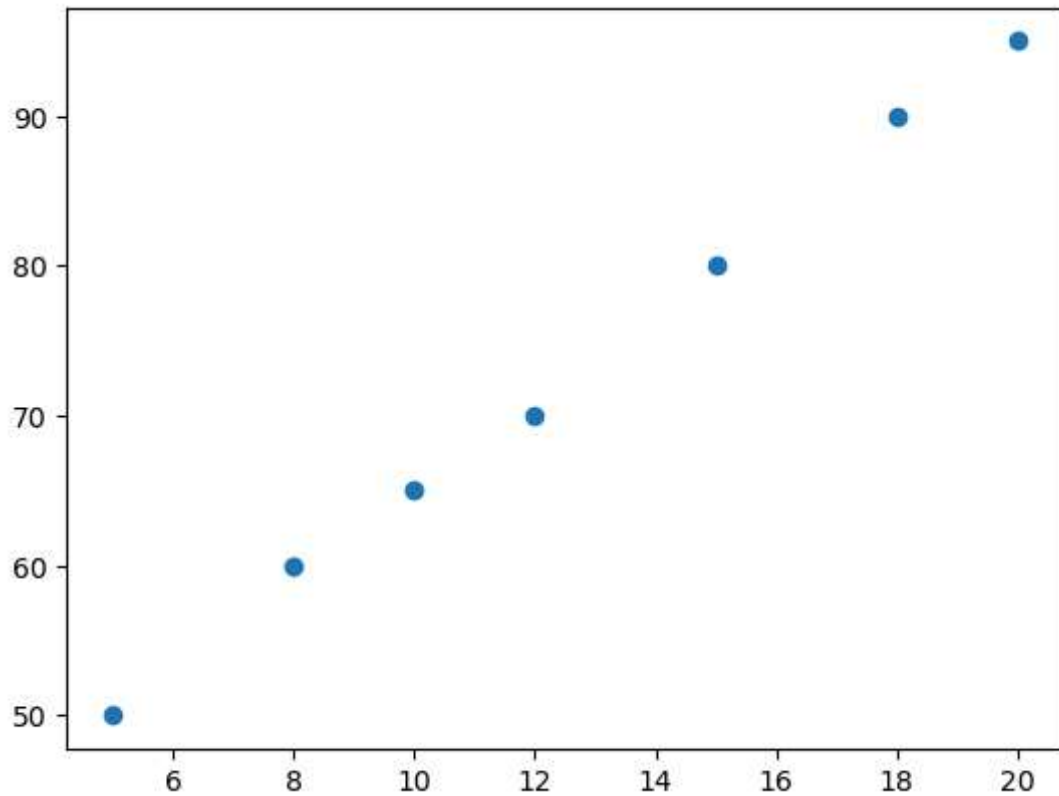
```
In [20]: l=['1st','2nd','3rd','4th','5th']  
c=['b','g','r','c','m']  
w = [0.5,0.6,0.3,0.4,0.7]  
plt.title('No of books sold')  
plt.barh(Categories,Books_Sold,color=c,label=l)  
plt.show()
```



08) WAP to create a scatter plot representing the relationship between advertising expenditure and sales revenue of a company.

```
In [3]: Advertising_Cost = [5, 8, 10, 12, 15, 18, 20]
Sales_Revenue = [50, 60, 65, 70, 80, 90, 95]
```

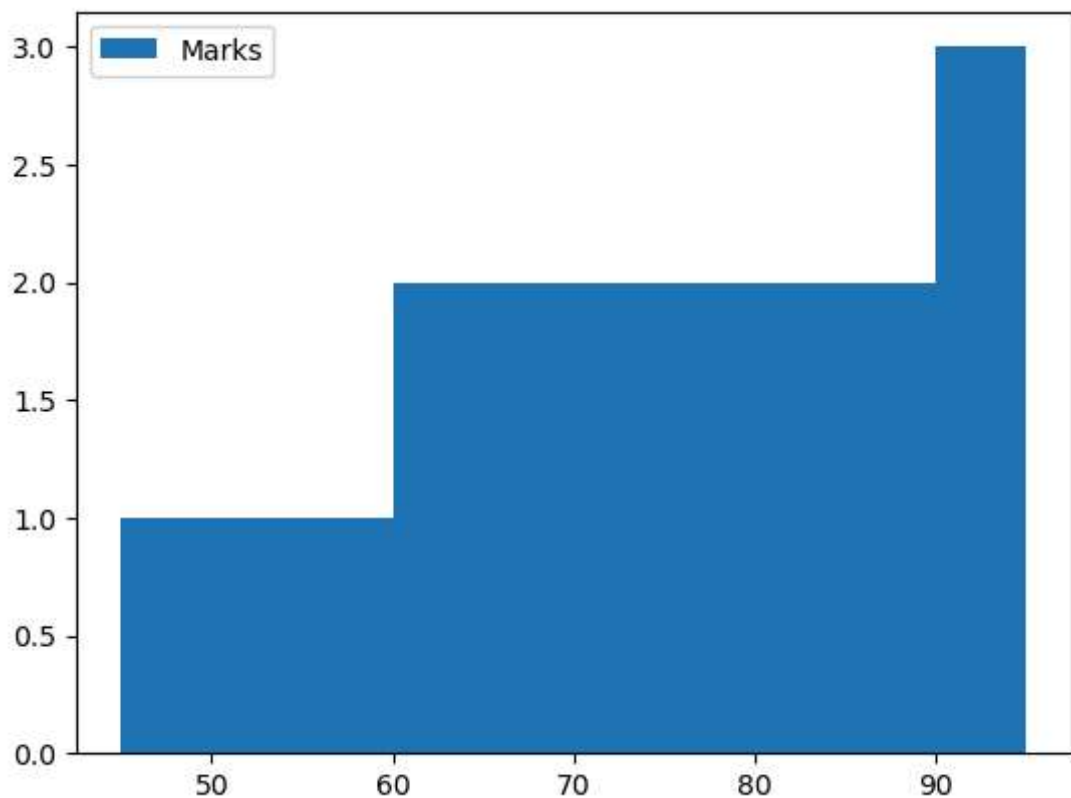
```
In [4]: plt.scatter(Advertising_Cost, Sales_Revenue)
plt.show()
```

09) WAP to create a histogram representing the distribution of marks obtained by students in a class.

```
In [5]: Marks = [45, 50, 55, 60, 62, 65, 68, 70, 72, 75, 78, 80, 82, 85, 88, 90, 92, 95]
```

```
In [6]: plt.hist(Marks, bins=10, histtype='stepfilled', align='mid', label='Marks')  
plt.legend()  
plt.show()
```



10) WAP to create a box plot representing the distribution of daily commuting time (in minutes) of employees and identify outliers (if any).

```
In [2]: Commuting_Time = [5, 18, 20, 22, 25, 27, 28, 30, 32, 35, 38, 40, 42, 90]
```

```
In [3]: plt.boxplot(Commuting_Time)  
plt.show()
```

